Veda-Core: An Object-Centric, Address-Less, Capability-Based RISC-V Extension for Deterministic Memory Safety

Author: Prabhudasu Vatala (independent researcher)
Repository: https://github.com/prabhu-euro20/Veda-Core
Status: Design-Stage Core with a complete formal model and RTL implementation; RTL-simulation and Sail-execution results -- no silicon or FPGA exists.

Abstract

Veda-Core is a RISC-V custom extension that removes the raw memory address as a software-visible concept and replaces it with an Object_ID: a system-wide handle resolved, at bind time, through a flat Object Descriptor Table (ODT) into a 128-bit hardware capability register carrying bounds, permissions, a type tag, and a generation counter. Every subsequent access through that capability is checked in hardware -- bounds, permissions, tag validity, and staleness -- before it reaches memory. I built and verified this architecture at two independent layers: a complete Sail formal model (30/30 self-checking tests) and a from-scratch TL-Verilog RTL implementation (27/27 milestone regressions, plus 51/51 on RISC-V International's own ACT4 RV64I conformance suite with zero regressions). I report real, measured results from both layers: a deterministic tag-validity guarantee (P(bypass)=0) against Arm MTE's probabilistic 4-bit tagging (up to 96% attacker success by 50 retries); a hardware-checked protected-return instruction (OCJALR) that is simultaneously safer and ~30% cheaper than the software-checked sequence it replaces; an object-descriptor-construction fast path (VEDA_ODT_POPULATE_FAST) that cuts per-object setup cost by 32.5%–40%, a ~6.5x larger improvement than the best software-only workaround for the same RV64I immediate-encoding limitation; a synthesis-based finding that the full per-access capability-check chain is *shorter*, not longer, than a plain load's address computation (95 vs. 114 logic-gate levels); and five real, RTL-executed attack demonstrations (out-of-bounds read/write, return-address hijack, use-after-free, pointer forgery) in which an unmodified RV64I core fails silently and Veda-Core hard-traps with the exact, verified cause code. I also report two real security bugs found and fixed during own development -- an ODT index-aliasing collision and a generation-counter ABA wraparound -- and I state plainly what is not yet built: no

silicon, no compiler toolchain, single-hart only, and a measured +20% dynamic-activity overhead. I position Veda-Core precisely against CHERI (the closest real precedent) using a comprehensive October 2025 literature survey of the object-aware-memory landscape, and invite critical feedback specifically from researchers with CHERI or capability-architecture experience.

1. Introduction

Memory-safety vulnerabilities remain the dominant class of real-world security defects: Microsoft attributes roughly 70% of the CVEs it patches to memory-safety issues, a figure repeated across the security industry [OpenSSF25]. The bug class has not gone away even in heavily-hardened, actively-maintained software — Chromium logged 205 CVEs in 2025 with use-after-free remaining the dominant bug class, and V8 alone has had multiple 2024–2025 CVEs chaining heap corruption or type confusion into sandbox escape — CVE-2024-7965 (heap corruption granting arbitrary read/write) and, actively exploited in the wild, CVE-2025-5419, CVE-2025-10585, and CVE-2025-13223.

Two broad hardware responses exist today. CHERI extends ordinary pointers with bounds, permissions, and an out-of-band tag, while keeping the underlying flat address space [CHERI-ISA]. Arm's Memory Tagging Extension (MTE) attaches a narrow, 4-bit color to memory and to pointers, checked on access — real, shipping, but **probabilistic**: an attacker who can retry succeeds with probability 1-(15/16)^k, which exceeds 96% by 50 attempts, a real and unremarkable number of retries for an automated exploit against a resettable service.

Veda-Core takes a third position, one the field explored in the 1970s–1980s (Plessey System 250, the Cambridge CAP Computer) and then broadly abandoned in favor of the flat, address-based model RISC and cost/compatibility pressure made dominant [DTong25]. I revisit it under a different, explicitly stated optimization target -- determinism and hardware-enforced memory safety, not raw throughput -- and ask whether the branch the field left behind is worth reopening now that its original abandonment reasons (1970s–80s silicon cost, ABI compatibility with an existing pointer-based C ecosystem) do not bind the same way for a new, purpose-built extension.

Concretely: software never constructs, computes, or holds a pointer in Veda-Core. It holds an Object_ID. A capability register is *bound* to that ID -- populated from a system-wide, flat Object Descriptor Table -- before any access, and every subsequent dereference is checked entirely in hardware, locally, using fields already cached in the capability register.

This report documents what is real and verified today: a complete instruction set specification, a formal Sail model, a working RTL implementation, and a substantial empirical evaluation across performance, security, and synthesis-level cost. We state plainly what remains open. We are seeking critical technical feedback, not announcing a finished system.

2. Related Work

CHERI [CHERI-ISA] is the closest real precedent and the primary comparison point throughout this report. CHERI capabilities are *pointer-shaped*: bounds, permissions, and a tag ride along with a real address, and CHERI's own official taxonomy work classifies this as a "Direct Pointer Capability" (bounds embedded directly in the address) [DTong25, Fig. 2, category c3]. CHERI-RISC-V is real, industrially backed (Google chairs the SIG/Task Group; Microsoft Research built CHERIoT; Codasip ships a conformant core; lowRISC ships the SONATA FPGA board), and still in draft: the live specification is version v0.9.9-draft, status "Stable" (one stage before "Frozen," two before "Ratified") as of 2026-07-21 — the Task Group's own "late summer 2025" ratification target has been missed by over ten months [CHERI-Summit25]. CHERI's own software-ecosystem cost is a real, quantified data point: 13 years, 1,800+ FreeBSD commits, three DARPA programs and £190M of UK government funding, with a CheriBSD Chromium port still "in progress" as of 2023 [Davis23].

Arm MTE and PAC are real, shipping mitigations. MTE's 4-bit tag gives a probabilistic guarantee (section 4.1); PAC is deployed on Apple Silicon and high-end Android SoCs but addresses control-flow, not general bounds, protection.

Cage (CGO 2025) is real, published, and directly relevant prior art I found via literature search rather than assuming the space was open: it uses Arm MTE and PAC -- both real, shipping hardware -- to add memory safety to WebAssembly sandboxing, with published results under 5.8% runtime overhead [Cage25]. This materially weakens any claim that hardware-accelerated software-sandbox safety is itself novel; our own differentiation against it is narrower and stated precisely in section 6.4.

Historical precedent. The Plessey System 250's System Capability Table is a real, shipped, flat, Object_ID-indexed table (589 KB for 64K descriptors at 16-bit width) [Levy84, ch.4]. The Cambridge CAP Computer's capability unit is populated by a backing Process Resource List -- structurally the model Veda-Core's Object-Bind mechanism follows, with one deliberate change: CAP's PRL was process-local, and CAP's own designers wrote a first-party regret about that choice ("the process tree

had been much overemphasized… led to performance and implementation difficulties") [Levy84, ch.5, section 5.12]. The Intel iAPX 432 made object-indirection *ambient* -- even scalar register operands paid a real indirection cost, and its CALL instruction cost 300 microseconds on early silicon, reduced to "under 100 microseconds" only after major rework, still roughly 3x slower than non-indirection-heavy contemporaries [Levy84, ch.9; Colwell88]. We treat this as a load-bearing negative precedent throughout our own design (section 3, section 7).

3. Architecture

3.1 Instruction set

Veda-Core reuses standard RV64I instruction formats and adds three custom instruction families under RISC-V's reserved custom-opcode space (Custom-0/1/2; Custom-3 deliberately left unallocated, following CHERI's own documented 13-year lesson of needing more encoding room than initially planned):

- **Custom-0 (OCL/OCS)**: address-less data movement (OCL.{B,H,W,D,C} / OCS.{B,H,W,D,C}, widths mirroring RV64I's own load/store funct3 table exactly), NMC_ADD.{W,D} (a memory-side compute-at-memory add, semantically AMOADD adapted to object-relative addressing, restricted to W/D widths mirroring RISC-V's own Zaamo restriction), and Object-Bind (Bind/Bind-NoTrap/Rebind).
- **Custom-1 (Veda-Atomic)**: nine AMO-style operations reusing RISC-V Zaamo's own op-select encoding verbatim.
- **Custom-2 (Veda-Cap)**: the query family (CGetBase/Len/Perm/Tag/Type/Addr/Offset), CSetBounds/CSetBoundsExact, OCA (offset adjustment, Veda-Core's CIncOffset equivalent), CSeal/CUnseal, OCInvoke (protection-domain transition, Veda-Core's CInvoke equivalent), and OSpecialRW (special-register read/write, Veda-Core's CSpecialRW equivalent).

Every instruction family we designed by direct, cited adaptation of a real CHERI-RISC-V instruction's own published semantics (CIncOffset, CSeal/CUnseal, CSetBounds, CInvoke, CSpecialRW, CJALR), not invented independently — a deliberate choice to reuse a mature, scrutinized design wherever the underlying operation is the same, and to depart from it only where our object-relative addressing model genuinely requires a different mechanism.

3.2 Capability register file

Sixteen 128-bit registers (c0–c15), each with a 1-bit out-of-band Tag. Field layout: Object_ID(23) + Base(32) + Length(16) + Offset(16) + Perms(16) + otype(16) + Reserved(8) = 127 bits + 1 padding bit = 128, plus the out-of-band Tag. Object_ID was widened from an initial 16 bits (65,536 objects) to 23 bits (8,388,608 objects) during development, freed by right-sizing the generation-counter field to 8 bits — a width chosen by direct cross-reference to CHERI-D's own empirically-derived, hardware-prototyped per-allocation generation-counter width [WangCHERID].

Base/Offset are not software-supplied: Base only ever exists in a register after a real ODT lookup, and Offset is the capability's own persistent, object-relative cursor ($0 \leq$ Offset $<$ Length; the real location is always Base + Offset). Software never constructs a raw address.

3.3 The Object Descriptor Table and Object-Bind

The ODT is a flat, single-level, system-wide table indexed directly by Object_ID, holding Base/Length/Perms/generation/owner_hart per entry. We chose flat-and-system-wide after reading the historical alternatives in full (seL4's guarded, multi-level CSpace; the Cambridge CAP's process-local PRL) rather than assuming: a multi-level table would make Object-Bind's latency depend on lookup depth, which conflicts with our deterministic-latency design goal, and CAP's own designers' first-party regret about process-local scoping is direct evidence against that choice specifically. The ODT is ordinary DRAM-resident memory, walked fresh on every Object-Bind with no caching layer of any kind — Veda-Core has no cache anywhere in its design, a deliberate choice discussed in §7.

Object-Bind (Bind/Bind-NoTrap/Rebind) is the only mechanism that populates a capability register from the ODT. Rebind is the concrete instruction that makes our headline design claim literal: it refreshes Base/Length/Perms/otype/generation from the ODT while leaving Offset untouched, so an object relocated by rewriting its ODT entry is transparently followed by software's own unchanged relative position. We proved this against a real physical address, not just capability metadata: an object was re-populated at a new Base in RTL, and a capability register's own untouched Offset correctly addressed the new physical location on the very next access (RTL Milestone 8).

Temporal safety. Each ODT entry carries a generation counter, cached into a capability at bind time and re-checked against the live ODT entry on every dereference; a mismatch — the object having been destroyed and its slot reused since binding — is a Tag Violation. This mechanism is our own synthesis: the

historical literature we read in full names exactly two solutions to the dangling-reference problem (never reuse an identifier; find and disable every outstanding capability on delete) [Levy84, ch.10], and a generation counter is neither. We state this as a narrower guarantee than general capability revocation, not overclaimed: it detects staleness after ID reuse, uniformly for every outstanding capability, not selective per-holder revocation.

Concurrency policy. Object-Bind is exclusive-by-default: an ODT entry records the owning hart, and a plain Bind against a live object owned by a different hart hard-traps. This policy is grounded in the only publicly available, commercially shipping real-world processing-in-memory hardware we could find: UPMEM's DPU system has no inter-DPU communication channel at all, and its own official programming guidance is to split workloads into independent blocks specifically to avoid this exact contention class [GomezLuna22]. We have not built or tested real, concurrent multi-hart RTL; owner-hart enforcement is verified via direct ODT-state injection standing in for a second hart, stated honestly in §8.

3.4 Trap model and protection-domain transitions

A single custom exception code (mcause = 0x18, RISC-V's own "designated for custom use" range) covers every Veda-Core violation, with an mtval layout (cap_idx + cause) matching CHERI-RISC-V's own real xtval format bit-for-bit. We drew a deliberate, verified split between two categories of instruction, found by reading four independent real CHERI instruction definitions in full rather than assuming one uniform rule: instructions that *manipulate* a capability's own metadata (OCA, CSetBounds, CSeal/CUnseal, Rebind) soft-fail (clear the Tag, no trap), matching CHERI's own clearTagIf convention; instructions that *use* a capability to actually dereference an object or transfer control (OCL/OCS, Veda-Atomic, OCInvoke, OCJALR) hard-trap.

OCInvoke is our CInvoke equivalent: two sealed capabilities (code, data) with matching type, correct Permit_Invoke/Permit_Execute permissions, atomically unseal and redirect execution, installing the unsealed data capability at a fixed register index (c15, proportional to CHERI's own C31 — itself just the last entry in CHERI's register file, not a separate physical register, a fact our own research corrected from an earlier assumption). We deliberately do not maintain a full CHERI-style Program-Counter Capability (PCC): CHERI's PCC exists because pure-capability CHERI makes *every* instruction fetch capability-checked, a choice Veda-Core never made (fetch stays plain RV64I). Instead we built the narrower, real gap: OCInvoke's own success path narrows two persistent registers to the invoked capability's bounds, and every subsequent fetch is checked against them, hard-

trapping on escape — genuine fetch-time compartment enforcement without a universal, always-on PCC.

OCJALR closes a gap we found by testing our own design, not by inspection: a return-address-protection convention built entirely from already-existing instructions (derive and seal a return capability via OCA+CSeal; verify and unseal on return via OCL.C+CGetTag+CUnseal+CGetAddr) worked correctly only if the programmer remembered an explicit tag check before jumping — nothing in hardware stopped that check from being silently omitted. We proved this concretely: a corrupted return capability used *without* the explicit check produced an undefined jump target, not a safe failure (§6.5). OCJALR merges verify-and-jump into one atomic, hardware-enforced instruction, adapted from CHERI's real CJALR.

4. Verification Methodology

We built and verified Veda-Core at two independent layers, in that order — catching specification bugs in an executable formal model before RTL exists to debug against, the same rationale CHERI's own sail-cheri-riscv effort documents for itself.

Formal model. We extended our own already-built sail-riscv checkout with a new model/extensions/Veda/ directory, using the same native per-extension mechanism every real RISC-V extension in that tree uses (not a fork of CHERI's own sail-cheri-riscv, whose flat-address, MMU-integrated capability format would fight our object-relative model). The model implements the complete instruction set described in §3, compiled into a real sail_riscv_sim binary. **30/30 self-checking positive/negative tests pass**, using the model's own real, built-in HTIF/tohost support and a reusable trap-handler pattern that asserts the exact expected mcause/mtval, not merely that some trap fired.

RTL. TL-Verilog (SandPiper → SystemVerilog → Icarus Verilog), built as veda_core.tlv, layered on our own RVA23-conformant base core (rv64i_core.tlv, single-cycle, all 50 RV64I encodings, independently 51/51 on ACT4). Eighteen sequential milestones took Veda-Core's RTL from a capability register file and Object-Bind through the full instruction set, real Zicsr-lite trap infrastructure, owner-hart enforcement, PCC-equivalent compartment bounding, and two post-hoc security-hardening passes (§5). **27/27 milestone regression tests pass.**

Base-ISA conformance. We ran RISC-V International's own official ACT4 RV64I conformance suite — the same 51-test corpus our unmodified base core already

passes — directly against veda_core.tlv itself, not a proxy file, after all eighteen milestones landed. **51/51 pass, zero regressions**, including after Milestone 14 added a check that unconditionally forces the fetched instruction word to a substitute value under one condition — the single highest-blast-radius change in the file, now independently confirmed side-effect-free against a broad, externally-authored, reference-signature-checked corpus.

Formal-verification maturity, stated honestly. We used Sail's own official Coq/Rocq export backend to translate the complete model (630,731 + 5,430,435 bytes, ~118,761 lines of generated Coq) under Sail's strictest type-checking flags, and confirmed our two real bug fixes (§5) translate correctly and faithfully into the generated code. This proves the model translates to valid Coq syntax under strict typing; it does not prove anything is formally *correct* — no Coq compiler is installed in our environment, no proof obligation has been written or checked. We calibrate this honestly against the real, mature precedent: the sail-cheri-riscv codebase itself is 51.3% Isabelle, 45.9% Rocq/Coq, and only 2.6% executable Sail — our own executable-model-plus-tests work sits entirely in that smallest 2.6% slice of what "formally verified" means at CHERI's own maturity level.

5. Real Bugs Found and Fixed

We report two genuine security-relevant RTL bugs found by empirical testing during our own development, not merely as an abstract discussion of risk classes.

ODT index aliasing (fixed, RTL Milestone 15). Our RTL's 256-entry ODT indexed only the low 8 bits of the real 23-bit Object_ID. We reproduced the consequence directly: populating Object_ID=100 and Object_ID=356 (same low byte) caused a fresh Bind for Object_ID=100 to silently resolve to the other object's data — a real confused-deputy/corruption primitive, not a theoretical capacity limit. Fixed by storing the requested ID's upper 15 bits in previously-unused ODT-entry space and checking it on every lookup; a collision now reads as "not found" and plain Bind hard-traps. Verified via a real negative reproduction, a positive control, the full 23-test milestone suite, and the full 51-test ACT4 suite, all clean. This bug was RTL-only; the Sail model's own ODT was already declared over the full 23-bit space with no truncation.

Generation-counter ABA wraparound (fixed, both layers). The 8-bit generation counter wraps after 256 ODT-Destroy operations on one slot. We reproduced this directly: destroying the same Object_ID 256 times, then re-populating it, made a capability cached *before* the very first destroy pass its staleness check again — a textbook ABA-problem use-after-free false negative,

now concretely demonstrated on real RTL. We considered and rejected the obvious fix (saturate at 0xFF) because it makes every future incarnation of that slot permanently indistinguishable rather than only periodically so; the real fix permanently retires a slot once its generation would wrap. This deliberately strengthens Veda-Core beyond CHERI-D's own published position, which treats generation wraparound as a statistically-rare, accepted residual risk derived from real allocator-reuse traces [WangCHERID] — we judged a real, demonstrated exploit on our own hardware a stronger basis for closing the gap than a statistical argument for accepting it, given our own explicitly stated security-over-throughput priority. Mirrored into the Sail model once we located an already-installed Sail toolchain in our own environment; both layers verified clean (27/27 RTL, 26/26 Sail at the time).

Beyond these two, our milestone-by-milestone development record documents numerous smaller, honestly-distinguished bugs — a partial-field-copy bug in OCA's first RTL draft, a genuine byte-granular tag-invalidation gap found only after OCL.C/OCS.C existed (a plain write to a previously-tagged memory granule must clear that granule's tag, or a corrupted capability's bytes could be silently read back as still valid), and a bind-mode field that was decoded since the very first RTL milestone but never actually checked until Milestone 8, meaning Rebind silently executed as plain Bind for seven milestones. We report this development history because we consider it evidence of methodology, not something to omit: build a minimal slice, run it for real, and treat a passing test as the only real evidence.

6. Evaluation

All numbers below are read directly from real Icarus Verilog simulation of the actual, unmodified, committed RTL, or from real Sail execution — not projected or estimated, unless explicitly marked as an external, cited source.

6.1 Deterministic vs. probabilistic tag validity

Arm MTE's 4-bit tag gives an attacker who can retry a success probability of 1-(15/16)^k — 6.25% at one attempt, 96.03% by 50, 99.84% by 100 [Cage25, cited MTE analysis]. Veda-Core's Tag is not a color-match scheme: it is a single out-of-band bit that can only be set by a real, authorized capability-producing instruction, never by an ordinary data write, under any circumstances. We confirmed this empirically (§6.5, Demo 5): an attacker who fully controls 128 bits of forged data still produces Tag=0 unconditionally — a structural impossibility for that attacker

capability, not a low-probability event. P(success after k attempts) = 0 for any k, for the pure memory-corruption attacker model our demonstrations use.

6.2 Object-descriptor-construction cost, and a real ISA fix outperforming its software workaround

RV64I's 12-bit immediate limit forces plain ODT-Populate's packed 64-bit descriptor to be built via a full 6-instruction li sequence. A software-only workaround (loading the base via la instead) saved only ~5% at N=4 in our own measurement, because the base still had to be shifted into the descriptor's packed upper bits regardless of load mechanism. We built a genuine ISA-level fix instead — VEDA_ODT_POPULATE_FAST, which takes the base as a direct, unpacked operand and reads the reusable length/permissions half from a new persistent CSR set once via ordinary csrw. Measured against the real, unmodified core:

N	Plain ODT-Populate (cycles)	POPULATE_FAST (cycles)	Savings
1	10	9	10.0%
4	40	27	32.5%
16	160	99	38.1%

Exact closed form: 10N vs. 6N+3, a real, objdump-verified 40% per-object instruction-count reduction with no crossover point — it wins from N=1. The real ISA fix delivers a ~6.5x larger improvement than the best achievable software-only workaround for the identical problem, direct empirical confirmation that this specific overhead genuinely required a hardware fix.

6.3 Protection-domain crossing

Two independent comparisons. Against an external, cited real-world baseline: real, direct Linux context-switch measurements cluster around 1.2–2.2 microseconds on modern hardware [Bendersky18] — at plausible clock frequencies for such a system, on the order of 1,000–2,000 CPU cycles — with cache/TLB-pollution indirect costs pushing a widely cited practical rule of thumb to roughly 30 microseconds once real working-set sizes are accounted for [Sigoure10], a figure consistent in kind with an independent, peer-reviewed measurement of indirect context-switch cost via cache-refill effects [LiDingShen07]. OCInvoke executes in 1 instruction, 1 cycle on our own single-cycle core, because the target compartment's objects live in the *same* address space, structurally requiring no TLB flush or page-table switch — a 1,000x–2,000x difference in raw cycle count for the specific operation of moving into a different

trust domain, though this is not a claim that OCInvoke performs everything a full OS context switch does (it involves no scheduler).

We also built a same-methodology, own-hardware benchmark: OCInvoke against the cheapest real thing plain RV64I can do to gate a call at all (a software permission check before jal), both run on our own two real cores:

N	Traditional, software-gated (cycles)	OCInvoke (cycles)
1	10	41
8	73	62
16	145	86

Exact closed form: 1+9N vs. 38+3N — a real 3 cycles/crossing steady-state cost against 9 cycles/crossing for the software gate (3x per-crossing improvement), with a real, computed crossover at N≈6.17 including OCInvoke's own one-time setup cost.

6.4 Return-address protection

We tested three real programs against our own committed RTL: a plain RV64I sd ra/ld ra/jalr sequence with a simulated stack-buffer overflow (hijacked cleanly to an attacker-chosen address, x30=0xbad1); an object-centric protected convention built from already-existing instructions *with* an explicit tag check (prot_caught: corruption correctly caught, x30=0xca11); and the identical convention *without* the check (prot_gap: neither a clean hijack nor a safe failure — the corrupted capability's address field was read and jumped to unconditionally, producing an undefined target, PC and marker register both X-propagating). This is the real, honest finding that motivated OCJALR: capability metadata alone does not automatically prevent misuse of a corrupted-but-untagged value; without a hardware gate, protection was a software discipline, not a guarantee. Re-running the identical prot_gap scenario with the vulnerable tail replaced by a single ocjalr instruction and **no explicit software check written anywhere in the file** now produces a real, controlled hard-trap (x30=0xca11) instead of an undefined jump — the gap closed structurally, not by discipline. OCJALR measures 7 cycles/call steady-state against a 10-cycle software-checked equivalent (~30% cheaper) and against 5 cycles/call for the unprotected traditional convention.

6.5 Five real attack demonstrations

Each demo injects the same off-by-one or corruption bug on both the unmodified base RV64I core and Veda-Core, and reports real register/memory state read directly from simulation.

Demo	Traditional result	Veda-Core result
Out-of-bounds read	Secret leaked: x7=0xdeadbeefcafebabe	Trap, mtval cause=0x01 (Bounds), secret never read
Out-of-bounds write	Canary overwritten: 0xbad0bad0bad0bad0	Trap, canary untouched
Return-address hijack	x30=0xbad1, control-flow hijacked	Trap via OCJALR, x30=0xca11
Use-after-free	x7=0xbbbbbbbbbbbbbbbb, object B's data returned through a stale reference to object A	Trap, mtval cause=0x02 (Tag Violation, stale generation)
Pointer forgery	Forged 128-bit pattern used unconditionally as a working pointer, x7=0xdeadc0dedeadc0de	CGetTag=0 on the forged value; subsequent use hard-traps

We state plainly what this does and does not show: the base-ISA behavior demonstrated is real and transfers directly (no mainstream ISA's raw load/store has bounds, tag, or use-after-free checking built in), but the traditional core carries zero of the additional real mitigations production systems sometimes layer on top (MMU paging, Arm PAC, Intel CET). MMU/paging-based protection is real but operates at page granularity (typically 4 KiB) — standard, textbook operating-systems and computer-architecture knowledge, not a claim requiring a single citation — and every demo here is an intra-page, same-page overflow, the exact class page-table-based protection cannot address regardless of configuration. This is the same argument CHERI's own literature makes for its value proposition against the same real mitigations; we use the identical, community-accepted framing rather than a new one.

6.6 Synthesis-level cost

Using Yosys (technology-independent, no PDK available in our environment — a generic gate-level signal, not an absolute-picosecond one), we synthesized OCL.D's complete real check chain (Tag, generation-staleness ODT re-read, Seal, Permission, Bounds, address computation) as a faithful transcription of the actual RTL expressions, against a plain RV64I load's own address computation:

	Longest topological path (gate levels)	Total mapped cells
Plain RV64I load address	114	351
Veda-Core OCL.D full check + address	95	233

The checked path is *shorter*, not longer — verified across two different ABC mapping strategies giving identical results. Two real, structural reasons: the security checks run in parallel with, not serially before, the address computation (only the downstream write is gated), and a capability's Base field is a genuine 32 bits (zero-extended), narrower than a traditional core's full 64-bit register value, letting synthesis exploit a constant-zero upper half. This meaningfully de-risks, without resolving, the single largest previously open question in our own evaluation program: whether the capability-check logic threatens maximum clock frequency on a pipelined implementation.

6.7 Register pressure and energy

Capability-register working-set pressure, k distinct objects round-robin across 16 physical registers: overhead is flat (1.186x–1.187x) at or under the real 16-register capacity, regardless of whether k is 8 or 16, then rises in proportion to how far the working set exceeds capacity — 1.231x at k=17 (one aliasing pair), 1.561x at k=32 (full 2x oversubscription, our worst measured case). We note this worst case is still under half of the iAPX 432's own real, documented, post-rework overhead (~3x) [Levy84, ch.9].

Using a real VCD signal-toggle count as a standard, if coarse, first-order dynamic-power proxy (no PDK exists for an absolute power number), Veda-Core's per-cycle activity is ~20% higher than the traditional core's — real, and, after normalizing for both extra hardware and extra cycles, explained mostly by Veda-Core simply having more real hardware present (a genuinely unconditional, every-cycle PCC-bounds check among the contributors), not by that hardware being disproportionately active. This is the one place our own two stated design goals — security and energy efficiency — are not both currently met, stated as such rather than glossed over.

6.8 Object-centric access, baseline cost

A sequential array-sum benchmark (bind once, access N elements) against the traditional core, both on our own real single-cycle RTL: traditional_cycles = 4+5N, veda_cycles = 14+5N — a fixed +10-cycle one-time setup cost, zero additional per-

access cost once bound, amortizing to under 2% overhead by N=64 and under 1% by roughly N=256 (extrapolated from the exact closed form).

7. Positioning Against CHERI and the Current Literature

We deliberately checked our own uniqueness claim against the most current, comprehensive mapping we could find rather than asserting it from our own prior research alone: a 162-reference arXiv survey of the entire descriptor/capability/object-aware memory landscape, posted October 2025 [DTong25], read in full — a preprint, not a peer-reviewed publication, a distinction we state explicitly rather than implying a review process this work has not gone through. Its own capability taxonomy independently confirms the classification our own historical reading (Plessey, Cambridge CAP) had already reached: Veda-Core sits in category (c2), *Indirect Pointer Capability* (Object_ID + rights + a re-derivable address), distinct from CHERI's own (c3), *Direct Pointer Capability* (bounds embedded directly in the address). The survey's own historical timeline places "Capability-Based Architecture" as a 1970s–1980s-era category with no active 2000s–2020s branch; every modern system it covers in depth (CHERI, Intel MPX, Low-Fat, HotBound, and others) is address-based metadata, operating on top of the flat-address-space paradigm the field converged on for cost, simplicity, and compatibility reasons [DTong25, §3.3]. We read this as real, current, independent evidence that Veda-Core's specific combination — zero raw addresses anywhere at the ISA level, zero cache layer, one unified object-table mechanism gating both access and compute dispatch, opt-in atop a standard RV64I base, explicitly not optimizing for throughput — is a deliberate revival of a branch the field left for reasons tied to an optimization target we do not share, not a claim that no single mechanism here is precedented, and not a claim of superiority over CHERI's own real, differently-reasoned design (CHERI's own real constraint, ABI compatibility with an enormous existing C/pointer codebase, is one Veda-Core, as a from-scratch extension, does not carry).

We also checked our own explicitly stated "no cache" design pillar against the same survey: it treats making caches *object-aware* as a still-open future research direction, with every system it covers in depth assuming a conventional cache hierarchy exists [DTong25, §6.1]. Veda-Core's cache-less design has no counterpart in the surveyed literature, confirmed by what the field's own forward-looking agenda still treats as unresolved.

8. Honest Limitations

We state these plainly, matching the discipline we tried to hold throughout our own development record, not as a pro forma section.

- **No silicon or FPGA bitstream exists.** Every result in this report is RTL simulation or Sail execution — the same evidentiary tier real capability-architecture research, including CHERI's own early publications, used before silicon existed for it either.
- **Single-hart only.** Owner-hart enforcement (§3.3) is verified via direct ODT-state injection standing in for a second hart; neither our Sail simulator invocation nor our current RTL can produce genuine concurrent multi-hart execution. Real, physical multi-hart RTL remains substantial, unstarted work.
- **No compiler or software toolchain.** Every test program in this report is hand-assembled. CHERI's own real, decade-plus, multi-million-pound ecosystem effort (§2) is the honest floor for what a real software stack eventually costs — we do not claim to have replicated any of it, only the smallest first slice (formal model, RTL, conformance).
- **Energy overhead is real and unresolved** (§6.7): +20% per-cycle dynamic activity, the one place our own two stated design goals are not both currently met.
- **Formal-verification maturity is a small slice of what the term means at real maturity** (§4): an executable, tested Sail model and a syntax-valid Coq translation, not a single machine-checked theorem, against a real precedent where the executable model is only 2.6% of total verification effort.
- **OCJALR does not itself mint a fresh sealed return capability** (unlike CHERI's own general-purpose CJALR); the call site still needs its own, already-proven OCA+CSeal pair.
- **The 16-bit Length field caps a single object at 65,536 bytes** — a real, stated trade-off; growing past it needs either a wider capability register or a CHERI-Concentrate-style compressed bounds encoding, a separate design task we have not attempted.
- **No memory-encryption or remote-attestation mechanism exists** — Veda-Core's capability checks gate what instructions executing through the core's own checked path can do; they provide no defense against an attacker who can read raw DRAM directly (a malicious hypervisor with DMA, a physical attacker). We position Veda-Core as a complement to, not a substitute for, confidential-computing memory encryption (Arm CCA / Intel TDX / AMD SEV-SNP), addressing a different, real, currently under-addressed problem — memory-safety bugs *inside* the trust boundary those systems protect, the exact gap independent research (TeeRex, USENIX Security 2020) found still open across 8 real SGX/RISC-V/Sancus shielding frameworks even with Rust in the mix [Cloosters20].

9. Conclusion and What We Are Looking For

Veda-Core is a real, working, two-layer-verified (Sail + RTL) instantiation of an architectural branch the field explored decades ago and largely set aside — reopened here under a different, explicitly stated optimization target. Every quantitative claim in this report traces to a real, reproducible Sail or RTL run against the project's own committed source, listed with full detail and reproduction commands in the project repository. We are looking for critical technical feedback, specifically from researchers with direct CHERI or capability-architecture experience, on three questions we cannot fully answer from inside

our own project: whether removing the address from the ISA-visible interface entirely — rather than CHERI's own address-carrying capability — buys anything real beyond what we have measured, or is simply CHERI with an extra table-indirection; what prior art in the "pure Object_ID, no linear address" space we may not be aware of; and where our own Sail/RTL claims look overstated relative to what CHERI's own real, harder-won experience suggests is actually difficult (compiler/ABI integration, revocation cost, multi-hart ODT coherence).

References

[CHERI-ISA] RISC-V CHERI Task Group (Watson, R.N.M., Chisnall, D., Davis, B., et al., 40+ contributors, RISC-V International / University of Cambridge / SRI International / Google / Codasip). *RISC-V Specification for CHERI Extensions.* Living draft, version v0.9.9-draft as of 2026-07-29, status "Stable." https://riscv.github.io/riscv-cheri/ — the specific instruction-semantics pages cited throughout this report (CIncOffset, CSeal/CUnseal, CSetBounds, CInvoke, CSpecialRW, CJALR) refer to this live, RISC-V-International-hosted draft, not an earlier, standalone Cambridge technical report (UCAM-CL-TR-876/891/907/927/951), which covers an earlier, pre-RISC-V-International-standardization version of CHERI and uses different pagination.

[CHERI-Summit25] Kurd, T. (Codasip), Laurie, B. (Google). "Standardizing CHERI-RISC-V." RISC-V Summit Europe, Paris, May 14, 2025.

[DTong25] Tong, D. (Peking University). "Descriptor-Based Object-Aware Memory Systems: A Comprehensive Review." arXiv:2510.27070, submitted October 31, 2025, revised November 10, 2025. **Correction from an earlier internal draft of this report: this is an arXiv preprint, not an ACM-published or peer-reviewed work** — we cite it as a comprehensive, current literature survey, not as evidence of peer review.

[Davis23] Davis, B. "A Dozen Years of CheriBSD." FreeBSD Journal, May/June 2023 (30th Anniversary Special Edition).

[Levy84] Levy, H.M. *Capability-Based Computer Systems.* Digital Press, 1984, ISBN 0-932376-22-3.

[Colwell88] Colwell, R.P., Gehringer, E.F., Jensen, E.D. "Performance Effects of Architectural Complexity in the Intel 432." ACM Transactions on Computer Systems, Vol. 6, No. 3, August 1988, pp. 296–339.

[WangCHERID] Wang, Y., Woodruff, J., Mazzinghi, A., Rugg, P., Stark, S.W., Joannou, A., Watson, R.N.M., Moore, S.W. “CHERI-D: Secure and efficient inline object ID for CHERI temporal memory safety.” arXiv:2606.19055 (University of Cambridge).

[GomezLuna22] Gómez-Luna, J., El Hajj, I., Fernandez, I., Giannoula, C., Oliveira, G.F., Mutlu, O. “Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System.” IEEE Access, 2022.

[Cage25] Fink, M., Stavrakakis, D., Sprokholt, D., Chakraborty, S., Ekberg, J.-E., Bhatotia, P. “Cage: Hardware-Accelerated Safe WebAssembly.” Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO ’25), March 2025, Las Vegas, NV. arXiv:2408.11456.

[Bendersky18] Bendersky, E. “Measuring context switching and memory overheads for Linux threads.” eli.thegreenplace.net, 2018. (Reports 1.2–2.2 microseconds direct cost, depending on CPU affinity.)

[Sigoure10] Sigoure, B. “How long does it take to make a context switch?” blog.tsunanet.net, November 2010. (Source of the ~30-microsecond practical-worst-case rule of thumb once cache/TLB pollution is included; a distinct, separate claim from Bendersky’s own direct-cost measurement, not to be conflated with it.)

[LiDingShen07] Li, C., Ding, C., Shen, K. “Quantifying the Cost of Context Switch.” Proceedings of the 2007 Workshop on Experimental Computer Science (ExpCS ’07), ACM.

[Cloosters20] Cloosters, T., Rodler, M., Davi, L. “TeeRex: Discovery and Exploitation of Memory Corruption Vulnerabilities in SGX Enclaves.” 29th USENIX Security Symposium, 2020.

[OpenSSF25] “Announcing the Release of ‘The Memory Safety Continuum’.” Open Source Security Foundation blog, April 28, 2025.

A note on methodology: this project’s RTL, Sail model, tests, and this report were built with AI-assisted tooling (Claude, Anthropic) under close, iterative human direction. Every design decision, encoding choice, and measured result was independently reviewed and re-verified before inclusion here, and the standing discipline throughout has been to reject any claim not backed by a real, reproducible run. We state this plainly because verifiability, not authorship method,

is what should determine whether any of the above is worth a reader's time — every number in this report can be independently reproduced from the repository.